

Phase 1B Completion Summary: Core Infrastructure

Date: August 22, 2026
Version: 1.0.0
Status:  COMPLETE

Overview

Phase 1B established the core backend infrastructure for BaseCenter.ai, including FastAPI backend, PostgreSQL database, JWT authentication, and RESTful API endpoints.

Deliverables Completed

1. FastAPI Backend Structure

Location: /backend/

Architecture:

```
backend/
├── app/
│   ├── main.py                # FastAPI application entry point
│   ├── api/v1/
│   │   ├── auth.py           # Authentication endpoints
│   │   ├── users.py          # User management endpoints
│   │   ├── core/
│   │   │   ├── config.py     # Settings and configuration
│   │   │   ├── security.py   # JWT & password hashing
│   │   ├── db/
│   │   │   ├── database.py   # Database configuration
│   │   │   ├── models/
│   │   │   │   ├── user.py   # SQLAlchemy models
│   │   │   │   │   ├── user.py   # User model
│   │   │   │   │   ├── tenant.py # Tenant model
│   │   │   │   │   ├── module.py # Module model
│   │   │   │   │   ├── subscription.py # Subscription model
│   │   │   ├── schemas/
│   │   │   │   ├── user.py   # Pydantic schemas
│   │   │   │   │   ├── user.py   # User schemas
│   │   │   │   │   ├── token.py  # Token schemas
│   │   │   ├── services/
│   │   │   │   ├── user_service.py # Business logic
│   │   │   │   │   ├── user_service.py # User CRUD operations
│   ├── alembic/              # Database migrations
│   ├── tests/                # Test suite (placeholder)
│   ├── requirements.txt      # Python dependencies
│   ├── docker-compose.yml    # PostgreSQL Docker config
│   └── .env                  # Environment configuration
```

2. PostgreSQL Database

Configuration:
- **Database:** basecenter_db

- **User:** basecenter
- **Port:** 5432
- **Version:** PostgreSQL 18.6

Tables Created:

1. `users` - User accounts with authentication
2. `tenants` - Multi-tenant organizations
3. `modules` - Available business modules
4. `subscriptions` - Tenant-module subscriptions
5. `alembic_version` - Migration tracking

3. Database Models

User Model (`users`):

- id (Primary Key)
- email (Unique, Indexed)
- hashed_password
- full_name
- is_active
- is_superuser
- tenant_id (Foreign Key → tenants)
- created_at
- updated_at

Tenant Model (`tenants`):

- id (Primary Key)
- name
- subdomain (Unique, Indexed)
- stripe_customer_id
- is_active
- created_at
- updated_at

Module Model (`modules`):

- id (Primary Key)
- name (Unique, Indexed)
- slug (Unique, Indexed)
- description
- icon
- monthly_price (cents)
- is_active
- display_order

Subscription Model (`subscriptions`):

- id (Primary Key)
- tenant_id (Foreign Key → tenants)
- module_id (Foreign Key → modules)
- is_free_module
- stripe_subscription_id
- status
- current_period_start
- current_period_end

- created_at
- updated_at

4. Alembic Migrations

Setup:

- Alembic configured and initialized
- Initial migration created: 6ba3989c2e3f_initial_migration_users_tenants_modules_
- Database schema migrated successfully

Commands:

```
alembic revision --autogenerate -m "message"
alembic upgrade head
alembic downgrade -1
```

5. JWT Authentication

Implementation:

- **Library:** python-jose[cryptography]
- **Algorithm:** HS256
- **Access Token:** 30 minutes expiry
- **Refresh Token:** 7 days expiry
- **Password Hashing:** bcrypt

Security Features:

- Secure password hashing with bcrypt
- JWT token generation and validation
- Token-based authentication middleware
- Protected endpoint decorator

Fixed Issues:

- JWT "sub" claim must be string (converted from integer)
- Proper token decode error handling

6. Authentication Endpoints

POST /api/v1/auth/register

- Register new user
- Request: {email, password, full_name}
- Response: User object (no password)

POST /api/v1/auth/login

- Login with credentials
- Request: username (email), password (form-urlencoded)
- Response: {access_token, refresh_token, token_type}

GET /api/v1/auth/me

- Get current user profile
- Headers: Authorization: Bearer <token>
- Response: User object

7. User Management Endpoints

GET /api/v1/users/{user_id}

- Get user by ID
- Protected (authentication required)
- Authorization: Self or superuser only

PATCH /api/v1/users/{user_id}

- Update user profile
- Protected (authentication required)
- Authorization: Self or superuser only

8. CORS Configuration

Allowed Origins:

- http://localhost:3000 (Next.js dev server)
- https://c0ac1c01a.na111.preview.abacusai.app (Preview URL)
- https://basecenter.ai (Production)
- https://www.basecenter.ai (Production www)

9. Environment Configuration

Files:

- .env.example - Template with documentation
- .env - Active configuration (gitignored)

Variables:

- SECRET_KEY - JWT signing key
- DATABASE_URL - PostgreSQL connection string
- POSTGRES_* - Database credentials
- FIRST_SUPERUSER_* - Admin account
- API_V1_PREFIX - API versioning

10. Testing & Validation

Tested Flows:

1.  Root endpoint (/) - API info
2.  Health check (/health) - Status check
3.  User registration - Create new account
4.  User login - Get JWT tokens
5.  Protected endpoint - Access with token
6.  Database operations - CRUD working

Test Results:

- ✓ User registration successful
 - ✓ Login returns valid JWT tokens
 - ✓ Protected /me endpoint accessible with token
 - ✓ User data retrieved correctly
 - ✓ Password hashing working
 - ✓ Token validation working
-

Technical Stack

Backend

- **Framework:** FastAPI 0.115.0
- **Server:** Uvicorn 0.32.0
- **Language:** Python 3.11+

Database

- **Database:** PostgreSQL 18.6
- **ORM:** SQLAlchemy 2.0.36
- **Migrations:** Alembic 1.14.0

Authentication

- **JWT:** python-jose[cryptography] 3.3.0
- **Password:** passlib[bcrypt] 1.7.4 + bcrypt 4.2.1

Validation

- **Schemas:** Pydantic 2.10.3
 - **Settings:** pydantic-settings 2.6.1
-

API Documentation

Interactive Docs:

- Swagger UI: `http://localhost:8000/api/docs`
 - ReDoc: `http://localhost:8000/api/redoc`
 - OpenAPI JSON: `http://localhost:8000/api/openapi.json`
-

Database Commands

Start PostgreSQL

```
sudo service postgresql start
```

Access Database

```
sudo -u postgres psql -d basecenter_db
```

Run Migrations

```
cd backend
alembic upgrade head
```

Running the Backend

Install Dependencies

```
cd backend
pip install -r requirements.txt
```

Start Server

```
cd backend
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

Test API

```
# Root endpoint
curl http://localhost:8000/

# Register user
curl -X POST http://localhost:8000/api/v1/auth/register \
  -H "Content-Type: application/json" \
  -d '{"email": "test@example.com", "password": "pass123", "full_name": "Test User"}'

# Login
curl -X POST http://localhost:8000/api/v1/auth/login \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -d "username=test@example.com&password=pass123"
```

Known Limitations (Deferred to Later Phases)

1. **Email Verification:** Not implemented (manual activation only)
 2. **Password Reset:** Not implemented yet
 3. **Refresh Token Endpoint:** Token refresh not exposed
 4. **Rate Limiting:** No rate limiting on auth endpoints
 5. **Tenant Management:** Tenant creation/management endpoints not built
 6. **Module Seeding:** 10 business modules not populated yet
 7. **Stripe Integration:** Not connected (Phase 1C)
 8. **Super Admin Portal:** Not built (Phase 1D)
-

Next Steps: Phase 1C - Stripe Integration

1. **Stripe Setup:**
 - Connect Stripe account
 - Create products for 10 modules
 - Set up webhook endpoints
2. **Payment Endpoints:**
 - `/api/v1/subscriptions/create` - Subscribe to module

- `/api/v1/subscriptions/cancel` - Cancel subscription
- `/api/v1/webhooks/stripe` - Handle Stripe events

3. Business Logic:

- Module activation on payment
- Free module selection
- Subscription status tracking
- Email notifications

4. Testing:

- Stripe test mode integration
- Payment flow validation
- Webhook handling

Files Modified/Created

New Files (42 total)

- `backend/` directory structure
- All Python backend files
- Alembic migration files
- Docker Compose configuration
- Environment templates

Modified Files

- None (fresh backend implementation)

Git Commit

Branch: staging → main

Commit Message: "feat: Phase 1B - Core Infrastructure (FastAPI, PostgreSQL, JWT Auth)"

Changes:

- ☒ FastAPI backend with RESTful API
- ☒ PostgreSQL database with 4 core tables
- ☒ JWT authentication system
- ☒ User registration and login
- ☒ Alembic migrations
- ☒ Environment configuration
- ☒ CORS setup for frontend integration

Phase 1B Status: ☒ COMPLETE

Completion Date: August 22, 2026

Next Phase: Phase 1C - Stripe Integration
